Plataforma de Simulados Interativos do ENADE

Decisões técnicas e planejamento de sprints

Equipe
JORDAN FERNANDES DE CARVALHO
HENRIQUE VERAS CORDEIRO
MAYARA LIMA MIRANDA
IRAPUAM JUNIO DA SILVA SANTOS

Versão preliminar — novembro de 2026

1. Visão geral e premissas

Este documento consolida as decisões de ferramentas e o planejamento das sprints da Plataforma de Simulados Interativos do ENADE, a partir da EAP e do Product Backlog (PB01 a PB12) e do levantamento inicial de stack tecnológica elaborado pela equipe.

- **Escopo:** 12 histórias de usuário (PB01–PB12), organizadas em 5 épicos, somando 54 Story Points (SP). Os valores são sugestões do backlog; as estimativas finais serão definidas pela equipe no Planning Poker.
- **Processo:** Scrum, com sprints de aproximadamente uma semana e equipe de 4 integrantes.
- **Entrada de dados:** PDFs de provas predominantemente digitais, com texto selecionável (não digitalizações).
- **Princípio de projeto:** simplicidade. Uma aplicação principal em Next.js, banco PostgreSQL e um worker separado para tarefas demoradas de extração, sem microsserviços nem infraestrutura desnecessária.

> **Caráter preliminar:** toda a stack descrita aqui está sujeita a alterações. Trata-se de uma recomendação de partida, e nenhuma escolha deve ser considerada definitiva antes de testar as ferramentas com documentos reais e avaliar tempo disponível, familiaridade da equipe e necessidades que surgirem durante as sprints.

1.1 Épicos e estimativas

Épico	Histórias	SP
Importação e extração de provas	PB01, PB02	13
Revisão e curadoria	PB03, PB04	8
Aplicação do simulado	PB05, PB06, PB07	16
Autenticação e controle de acesso	PB08	5
Qualidade e desempenho	PB09, PB10, PB11, PB12	12
Total	**12 histórias**	**54**

1.2 Divisão da equipe em frentes

Frente	Histórias e atividades	Responsável
1. Frontend aluno	PB05; PB06 (telas); PB07; PB11	Mayara Lima Miranda
2. Frontend administrador	PB01 e PB02 (telas); PB03; PB04 (telas); PB12 (resposta rápida); tela de login (PB08)	Jordan Fernandes de Carvalho

Frente	Histórias e atividades	Responsável
3. Backend	PB08; PB12 (desempenho); APIs de PB03, PB04, PB05, PB06 e PB07; banco de dados; hash de senhas (PB10)	Henrique Veras Cordeiro
4. Tratamento de dados e extração	PB01; PB02; PB09; APIs de PB01 e PB02 (upload, fila, gabarito); preservação do PDF original (PB10)	Henrique Veras Cordeiro

1.3 Decisões sobre pontos não cobertos pela divisão original

Tema	Decisão
APIs de PB03 e PB05	Ficam com o Backend, junto às APIs de PB04, PB06 e PB07.
APIs de PB01 e PB02	Ficam com a frente de extração (upload, fila de processamento e importação do gabarito), por estarem ligadas ao worker.
PB10 — dados protegidos	Hash de senha com o Backend (via Better Auth). Preservação do PDF original com a frente de extração.
Tela de login (PB08)	Frontend administrador, sugestão para equilibrar a carga entre os frontends. Pode ser trocada para o Frontend aluno.

2. Decisões de stack tecnológica

A coluna **Situação** indica o estágio de cada decisão: **Adotada** ou **Recomendada** (ponto de partida do projeto), **Condicional** (depende de necessidade a confirmar) e **Opcional / Adiada** (pode ficar para depois).

2.1 Frontend e interface

Função	Ferramenta	Situação	Utilização no projeto
Framework web	Next.js	Adotada	Estrutura da aplicação, páginas, navegação e integração entre frontend e backend.
Biblioteca de interface	React	Adotada	Telas e componentes reutilizáveis usados pelo Next.js.
Linguagem	TypeScript	Adotada	Código tipado em frontend, backend, worker e regras de negócio.
Estilização	Tailwind CSS	Adotada	Construção e padronização dos estilos das telas.
Componentes	shadcn/ui	Adotada	Botões, diálogos, formulários, tabelas e navegação.

Next.js e TypeScript não são alternativas entre si: Next.js é um framework baseado em React, e TypeScript é a linguagem em que o código é escrito (.tsx para componentes, .ts para serviços, tipos e funções).

2.2 Backend, banco de dados e validação

Função	Ferramenta	Situação	Utilização no projeto
Endpoints da API	Next.js Route Handlers	Adotada	Recebem requisições do frontend. Evitam manter um servidor Express separado.
Banco relacional	PostgreSQL	Adotada	Usuários, perfis, edições, questões, alternativas, gabaritos, simulados e resultados.
ORM e migrações	Prisma ORM	Adotada	Modelos, consultas tipadas e controle de alterações do esquema.
Autenticação	Better Auth	Adotada (validar integração)	Login, sessões e autenticação por credenciais; hash seguro de senhas.
Validação	Zod	Recomendada	Valida dados de formulários e endpoints.

Função	Ferramenta	Situação	Utilização no projeto
Administração do banco	DBeaver Community	Opcional	Consulta visual de tabelas e registros.

A autorização por perfil (administrador e aluno) deve ser verificada no backend em cada operação protegida, e não apenas ocultando telas. Senhas nunca são guardadas em texto puro nem com criptografia reversível.

2.3 Importação, leitura e manipulação de PDFs

Função	Ferramenta	Situação	Utilização no projeto
Extração de texto	pdf-parse	Adotada (protótipo)	Extrai o texto dos PDFs digitais para identificar enunciados, alternativas e numeração.
Visualização no navegador	PDF.js (pdfjs-dist)	Condicional	Exibir a prova original ao administrador durante a revisão, se necessário.
Criação e edição de PDFs	pdf-lib	Condicional / adiada	Eventual exportação de resultados ou relatórios; não consta como história no backlog.
OCR	Tesseract.js (ou similar)	Não previsto	Somente se os testes revelarem arquivos digitalizados ou páginas sem texto selecionável.

O pdf-parse extrai texto, mas não identifica a estrutura lógica das questões. A frente de extração precisará implementar regras para localizar início e fim de cada questão, separar enunciado e alternativas, preservar a numeração original e sinalizar conteúdo que dependa de imagem, gráfico ou tabela. A extração textual é o primeiro caminho a testar.

2.4 Processamento em segundo plano

Função	Ferramenta	Situação	Utilização no projeto
Fila de tarefas	pg-boss	Recomendada (PB09)	Enfileira e gerencia o processamento das provas, usando o PostgreSQL como infraestrutura.
Execução	Worker Node.js/TypeScript	Adotada	Processo separado da aplicação web, no mesmo repositório, que executa a extração.

2.5 Armazenamento dos PDFs

Função	Ferramenta	Situação	Utilização no projeto
Desenvolvimento	Volume persistente do Docker ou diretório local controlado	Adotada	Mantém os PDFs entre reinicializações dos containers.
Remoto	Cloudflare R2	Opcional	Etapa posterior, se houver implantação remota. Bucket privado, com acesso controlado pelo backend.

O PostgreSQL guarda os metadados e a referência ao arquivo; o conteúdo binário do PDF fica no armazenamento de arquivos.

2.6 Testes e qualidade de código

Função	Ferramenta	Situação	Utilização no projeto
Unitários e integração	Vitest	Recomendada	Extração, associação de gabaritos, correção dos simulados e demais regras de negócio.
Ponta a ponta	Playwright	Recomendada	Fluxos completos: login, revisão de questões e realização de simulado.
Análise estática	ESLint	Adotada	Problemas comuns e padronização do código.
Verificação de tipos	TypeScript (tsc --noEmit)	Adotada	Erros de tipagem sem gerar arquivos de saída.

Os testes priorizam as regras que podem comprometer resultados: associação do gabarito à edição correta, questões anuladas, cálculo de acertos e erros, bloqueio de alterações após a finalização e permissões de cada perfil.

2.7 Versionamento, CI/CD, contêineres e hospedagem

Função	Ferramenta	Situação	Utilização no projeto
Controle de versão	Git	Adotada	Registro de alterações, branches e recuperação de versões.
Repositório e colaboração	GitHub	Adotada	Repositório remoto, Pull Requests e revisão de código.
Organização das tarefas	GitHub Projects	Opcional	Acompanhamento de histórias, tarefas e responsáveis.
CI/CD	GitHub Actions	Recomendada	Verificações automáticas em commits e Pull Requests. CD só se houver hospedagem.
Contêineres	Docker + Docker Compose	Recomendada	Ambiente local padronizado para toda a equipe.
Hospedagem	Render	Alternativa secundária (não confirmada)	Avaliar custos, limites, persistência do banco e armazenamento dos PDFs antes de adotar.

3. Arquitetura e fluxo de trabalho

3.1 Serviços do ambiente local

Serviço	Responsabilidade
web	Aplicação Next.js (telas e Route Handlers).
worker	Processo de extração e execução das tarefas da fila.
db	Banco PostgreSQL (também usado pela fila pg-boss).

O compose.yaml define variáveis de ambiente, rede entre os serviços e a persistência do banco e, se necessário, dos PDFs locais.

3.2 Fluxo de importação e extração

1. O administrador envia o PDF e informa a edição (ano e área).
2. A aplicação registra a edição e armazena o arquivo.
3. A aplicação cria uma tarefa na fila do pg-boss.
4. O worker obtém a tarefa e executa a extração com pdf-parse e as regras desenvolvidas pela equipe.
5. As questões identificadas são gravadas como pendentes de revisão.
6. O estado do processamento é atualizado para que o frontend mostre o andamento ou o resultado.

Falhas são registradas por questão, para que um problema isolado não interrompa as demais. A reexecução de uma tarefa não pode gerar registros duplicados. O limite de 3 minutos (PB09) não é garantido pela biblioteca de fila e deve ser medido com provas reais.

3.3 Regras de segurança e consistência

- Autorização verificada no backend em cada operação protegida.
- Questão não publicada não aparece para nenhum aluno em nenhuma tela.
- Correção calculada no backend; simulado finalizado não pode mais ser alterado.
- Questões anuladas ficam fora do cálculo do resultado.
- PDFs originais preservados para consulta posterior, com acesso controlado pelo backend.

3.4 Fluxo Git e integração contínua

- A branch main é a referência do código integrado; evitar alterações diretas nela.
- Cada responsável trabalha em branch própria ou por funcionalidade, e as alterações entram por Pull Request, com revisão de outro integrante sempre que possível.
- Em cada Pull Request para a main, o GitHub Actions executa: npm ci, ESLint, verificação de tipos, Vitest e build do Next.js.

Branches iniciais possíveis (nomes de exemplo):

```
main
├── feature/frontend-aluno
├── feature/frontend-admin
├── feature/backend-api
└── feature/pdf-extraction
```

4. Ferramentas por frente e pontos de integração

4.1 Ferramentas por frente

Frente	Responsável	Ferramentas mais diretamente relacionadas
Frontend aluno	Mayara Lima Miranda	Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Zod (formulários, quando aplicável), Playwright (fluxos do aluno).
Frontend administrador	Jordan Fernandes de Carvalho	Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, PDF.js (se houver visualização da prova original), Playwright (fluxos administrativos).
Backend	Henrique Veras Cordeiro	Next.js Route Handlers, PostgreSQL, Prisma, Better Auth, Zod, Vitest, Docker. Define os contratos de API dos dois frontends e integra o acesso do worker ao banco.
Extração de dados	Irapuam Junio da Silva Santos	TypeScript/Node.js, pdf-parse, pg-boss, worker separado, Prisma (persistência das questões), armazenamento local persistente (ou R2). PDF.js em conjunto com o frontend administrativo, se necessário.

4.2 Ferramentas e práticas compartilhadas

- Git e GitHub (branches e Pull Requests)
- Docker Compose (ambiente local comum)
- TypeScript, ESLint e verificação de tipos
- GitHub Actions; GitHub Projects, se a equipe optar por usá-lo

4.3 Pontos de integração entre as frentes

1. **Frontend e backend:** definir previamente endpoints, formatos de requisição e resposta e códigos de erro, para que o frontend trabalhe sobre o contrato enquanto o backend implementa.
2. **Frontend administrativo e extração:** o frontend envia o documento e consulta o estado do processamento, sem conhecer os detalhes do parser.
3. **Extração e backend:** combinar o modelo de dados de questões, alternativas, gabaritos, edições e estados de processamento; o worker persiste conforme esse modelo.
4. **Frontend aluno e backend:** o frontend apresenta e envia respostas, mas a correção é calculada no backend, e o resultado final fica protegido contra alterações.
5. **PB12:** o frontend administrativo cuida da interação de publicar/descartar; o backend garante operações pontuais e eficientes, sem carregar ou reprocessar todas as questões já revisadas.

5. Planejamento das sprints

O planejamento usa as cinco etapas do levantamento de stack como espinha dorsal, distribuídas em 6 sprints de aproximadamente uma semana. A distribuição de SP é indicativa e deve ser recalibrada no Planning Poker e conforme a velocidade real observada.

5.1 Visão geral

Sprint	Foco	Histórias	SP	Resultado esperado
1	Preparação e infraestrutura compartilhada	Tarefas técnicas	—	Todos executam a mesma base localmente e integram via GitHub.
2	Autenticação, modelagem e base das telas	PB08; PB05 (telas e API)	11	Login com acesso por perfil; contratos de API e modelo de dados definidos.
3	Importação e extração de provas	PB01; PB09 (parte)	11	Administrador envia uma prova e a extração roda em segundo plano.
4	Gabarito e curadoria	PB02; PB03; PB04; PB06 (telas)	15	Administrador importa o gabarito, revisa, publica ou descarta questões.
5	Aplicação do simulado e histórico	PB05 (integração); PB06; PB07; PB12	10	Aluno faz o simulado, vê a correção e consulta o histórico.
6	Qualidade e estabilização	PB09 (medição); PB10; PB11	7	Versão integrada, testada e estável.
Total		**12 histórias**	**54**	

5.2 Distribuição indicativa de SP por história e frente

História	Aluno	Admin	Backend	Extração	Total
PB01	–	3	–	5	8
PB02	–	2	–	3	5
PB03	–	3	2	–	5
PB04	–	1	2	–	3
PB05	5	–	3	–	8
PB06	2	–	3	–	5
PB07	2	–	1	–	3
PB08	–	1	4	–	5
PB09	–	–	–	5	5
PB10	–	–	2	1	3
PB11	2	–	–	–	2
PB12	–	1	1	–	2
Total	**11**	**11**	**18**	**14**	**54**

O Backend concentra a maior carga (18 SP), o que está registrado como risco na seção 6.

5.3 Detalhamento por sprint

Sprint 1 — Fundação

Story Points planejados: — (tarefas técnicas sem SP no backlog)

Objetivo: Preparar a base compartilhada do projeto (Etapa 1).

Frente / responsável	Entregas	SP
Frontend aluno [Mayara Lima Miranda]	Criar a aplicação Next.js com TypeScript Configurar Tailwind CSS e shadcn/ui Definir estrutura de diretórios e convenções do frontend	—
Frontend admin [Jordan Fernandes de Carvalho]	Criar o repositório no GitHub; definir fluxo de branches e Pull Requests Primeiros fluxos do GitHub Actions: lint, verificação de tipos e build	—
Backend [Henrique Veras Cordeiro]	Configurar Docker e Docker Compose; subir o PostgreSQL local Configurar o Prisma e criar os primeiros modelos do banco	—
Extração de dados [Irapuam Junio da Silva Santos]	Esqueleto do serviço worker no Compose Reunir provas e gabaritos reais do ENADE para os testes Definir o diretório ou volume persistente dos PDFs	—

Dependências e observações

- Repositório e aplicação base devem sair nos primeiros dias; as demais configurações se apoiam neles.
- Registrar essas tarefas no quadro da sprint, mesmo sem SP no backlog.

Resultado esperado: todos executam o mesmo projeto localmente e integram alterações por Pull Request, com o CI passando.

Sprint 2 — Autenticação e base das telas

Story Points planejados: 11

Objetivo: Usuários entram no sistema e acessam só o que o perfil permite; contratos de API e modelo de dados definidos (Etapa 2).

Frente / responsável	Entregas	SP
Frontend aluno [Mayara Lima Miranda]	PB05 (telas, 3 SP): listagem de edições, resposta por alternativa e navegação (avançar, retroceder, pular), com dados simulados Layout base e navegação do perfil aluno	3
Frontend admin [Jordan Fernandes de Carvalho]	PB08 (tela de login, 1 SP): credenciais inválidas e redirecionamento por perfil Estrutura de navegação do perfil administrador	1
Backend [Henrique Veras Cordeiro]	PB08 (4 SP): Better Auth, perfis administrador e aluno, autorização no backend Modelagem completa: usuário, perfil, edição, questão, alternativa, gabarito, simulado e resposta Contratos iniciais das APIs, documentados PB05 (API, 3 SP): listar edições com questão publicada e registrar respostas, com dados de seed	7
Extração de dados [Irapuam Junio da Silva Santos]	Spike do pdf-parse com provas reais (tarefa técnica) Definir com o Backend o modelo de questões, gabaritos e estados de processamento Decidir o armazenamento local dos PDFs	—

Dependências e observações

- Contratos de API e modelo de dados precisam fechar nos primeiros dias; o frontend aluno usa mocks até a API ficar pronta.
- O resultado do spike pode ajustar a estratégia de extração da Sprint 3.

Resultado esperado: login funcional, acesso restrito por perfil e contratos documentados.

Sprint 3 — Importação e extração de provas

Story Points planejados: 11

Objetivo: O administrador envia o PDF de uma edição e a extração ocorre em segundo plano, gerando questões pendentes de revisão (Etapa 3, parte 1).

Frente / responsável	Entregas	SP
Frontend admin **[Jordan Fernandes de Carvalho]**	PB01 (telas, 3 SP): envio do PDF, informe da edição (ano e área) e acompanhamento do estado do processamento	3
Extração de dados **[Irapuam Junio da Silva Santos]**	PB01 (5 SP): API de upload, armazenamento do PDF e parser que identifica questões, enunciados, 5 alternativas e numeração original; sinaliza questões que dependem de imagem, gráfico ou tabela PB09 (parte, 3 SP): fila pg-boss, worker separado, estado do processamento, falhas registradas por questão e proteção contra duplicidade em reexecução	8
Backend **[Henrique Veras Cordeiro]**	Apoio à integração do worker com o banco Testes Vitest de autenticação e permissões	—
Frontend aluno **[Mayara Lima Miranda]**	Ajustes da PB05 conforme o contrato Configurar o Playwright com o primeiro fluxo (login) Responsividade desde o início	—

Dependências e observações

- O frontend administrativo e a extração combinam o formato do estado de processamento.
- Backend e frontend aluno têm folga de SP nesta sprint e podem apoiar testes e revisão de código.

Resultado esperado: o administrador envia uma prova, a extração não bloqueia o uso do sistema e as questões ficam pendentes de revisão.

Sprint 4 — Gabarito e curadoria

Story Points planejados: 15 (sprint mais densa)

Objetivo: Importar o gabarito, revisar, publicar e descartar questões (Etapa 3, parte 2).

Frente / responsável	Entregas	SP
Frontend admin **[Jordan Fernandes de Carvalho]**	PB02 (telas, 2 SP): envio do gabarito vinculado à edição e indicação de questões anuladas PB03 (telas, 3 SP): lista de pendentes, edição de enunciado e alternativas, confirmação ou troca da alternativa correta PB04 (telas, 1 SP): publicar e descartar Avaliar o uso de PDF.js para exibir a prova original	6
Extração de dados **[Irapuam Junio da Silva Santos]**	PB02 (3 SP): importação do gabarito, associação automática da alternativa correta a cada questão e marcação de anuladas Ajustar as regras do parser com base nos problemas vistos na revisão	3
Backend **[Henrique Veras Cordeiro]**	PB03 (API, 2 SP): listar pendentes, editar, confirmar resposta PB04 (API, 2 SP): publicar e descartar, garantindo que questão não publicada não apareça ao aluno em nenhuma tela	4

Frente / responsável	Entregas	SP
Frontend aluno [Mayara Lima Miranda]	PB06 (telas, 2 SP): resultado com acertos, erros e gabarito por questão, com dados simulados	2

Dependências e observações

- PB02 depende da edição já importada na Sprint 3.
- PB04 precisa estar concluída antes da integração da PB05 na Sprint 5.
- Se a sprint estourar, as telas da PB06 são as primeiras candidatas a passar para a Sprint 5.

Resultado esperado: o administrador importa o gabarito, revisa as questões e controla quais ficam disponíveis aos alunos.

Sprint 5 — Aplicação do simulado e histórico

Story Points planejados: 10

Objetivo: O aluno realiza o simulado, finaliza e consulta correção e histórico (Etapa 4).

Frente / responsável	Entregas	SP
Frontend aluno [Mayara Lima Miranda]	PB05 (integração, 2 SP): trocar os dados simulados pela API real PB07 (telas, 2 SP): lista de simulados finalizados e reabertura do resultado Integrar a tela de resultado da PB06	4
Backend [Henrique Veras Cordeiro]	PB06 (API, 3 SP): finalização, cálculo de acertos e erros, anuladas fora do cálculo, gabarito por questão e bloqueio de alterações após finalizar PB07 (API, 1 SP): histórico do aluno PB12 (desempenho, 1 SP): publicar e descartar como operações pontuais, sem depender do volume já revisado	5
Frontend admin [Jordan Fernandes de Carvalho]	PB12 (resposta rápida, 1 SP): feedback imediato nas ações de publicar e descartar	1
Extração de dados [Irapuam Junio da Silva Santos]	Estabilizar o parser com as provas reais Testes Vitest das regras de extração e da associação de gabarito	—

Dependências e observações

- PB05 integra com questões publicadas reais, dependendo da PB04 concluída na Sprint 4.
- PB06 depende da PB05 e fecha nesta sprint, com telas e API.

Resultado esperado: o aluno responde, finaliza, vê acertos, erros e gabarito, e consulta resultados anteriores.

Sprint 6 — Qualidade e estabilização

Story Points planejados: 7

Objetivo: Entregar uma versão integrada, testada e estável (Etapa 5).

Frente / responsável	Entregas	SP
Frontend aluno [Mayara Lima Miranda]	PB11 (2 SP): finalizar a responsividade das telas do aluno em smartphones Testes E2E dos fluxos do aluno	2
Frontend admin [Jordan Fernandes de Carvalho]	Testes E2E dos fluxos administrativos Ajustes de usabilidade; decisão final sobre PDF.js	—

Frente / responsável	Entregas	SP
Backend **[Henrique Veras Cordeiro]**	PB10 (2 SP): validar o hash das senhas e o acesso aos PDFs originais pelo backend Revisar permissões e criar testes de integração	2
Extração de dados **[Irapuam Junio da Silva Santos]**	PB09 (restante, 2 SP): medir o tempo de extração em provas reais contra o limite de 3 minutos e testar falhas isoladas PB10 (1 SP): preservação do PDF original e consulta posterior	3

Dependências e observações

- Reservar tempo para correções de integração entre frontend, backend, banco e worker.
- Exportação de PDFs e hospedagem remota só entram se forem solicitadas.

Resultado esperado: versão integrada com os fluxos prioritários cobertos por testes unitários, de integração e ponta a ponta.

6. Riscos e pontos a validar

Risco ou ponto a validar	Como tratar	Quando
O pdf-parse extrai corretamente as provas reais escolhidas?	Spike com provas reais; se falhar, reavaliar a ferramenta ou considerar OCR.	Sprint 2
As regras de segmentação separam questões, enunciados, alternativas e numeração com precisão?	Iterar o parser com provas reais e sinalizar casos para revisão manual (imagem, gráfico, tabela).	Sprints 3 a 5
O pg-boss atende o fluxo e o worker cumpre o limite de 3 minutos?	Medir em provas reais, tratar reexecução sem duplicar registros e testar falhas isoladas.	Sprints 3 e 6
Há necessidade de R2, PDF.js ou pdf-lib?	Decidir com base na revisão (PDF.js) e nas solicitações da disciplina (R2, pdf-lib).	Sprints 4 a 6
O sistema será hospedado ou demonstrado localmente?	Decidir antes da Sprint 6; sem hospedagem, manter Docker Compose.	Sprint 5
Sobrecarga do Backend (18 SP, contra 11 a 14 nas demais frentes).	Frontend aluno e demais membros apoiam testes e validações nas sprints com folga (2 e 3).	Todas
Sprint 4 concentra 15 SP.	Se necessário, adiar as telas da PB06 para a Sprint 5.	Sprint 4
Frontend aluno depende de questões publicadas.	Usar mocks e dados de seed até a PB04 estar pronta.	Sprints 2 a 4
PB12 exige tempo de resposta que não cresça com o volume revisado.	Operações pontuais por identificador, índices adequados e medição com volume maior.	Sprints 5 e 6

7. Ritos e definição de pronto (proposta)

Seção sugerida para o grupo, sem origem nos materiais de partida. Pode ser ajustada ou removida.

Rito	Sugestão para sprints de uma semana
Planning	No início da sprint: escolher as histórias, estimar com Planning Poker e dividir as tarefas por frente.
Daily	Curta, de cerca de 10 minutos, ou assíncrona: o que foi feito, o que será feito e o que bloqueia.
Review	No fim da sprint: demonstrar o incremento funcionando.
Retrospectiva	Logo após a Review: o que manter, o que mudar, com ações para a sprint seguinte.

Definição de pronto sugerida para uma história:

- Critérios de aceitação do backlog atendidos.
- Pull Request revisado por outro integrante e integrado à main.
- CI verde (ESLint, tipos, Vitest e build).
- Testes automatizados para as regras de negócio da história.
- Contrato de API atualizado, quando houver mudança.

8. Pendências em aberto

- Recalibrar os SP de cada sprint após o Planning Poker.
- Confirmar a tela de login com o Frontend administrador (ou trocar para o Frontend aluno).
- Escolher as provas reais do ENADE que serão usadas nos testes de extração.
- Decidir sobre hospedagem remota, R2, PDF.js e pdf-lib conforme a seção 6.